Procedural Modeling in Verilog HDL

Digital Design and RTL Fundamentals

October 12, 2026

Contents

1 Introduction **3**

2 Basic Procedural Syntax **3**

3 The initial Block **4**
3.1 Example: Initial Block 4

4 The always Block **5**
4.1 Example: Always Block 5

5 Blocking Assignment **5**
5.1 Example: Blocking Assignment 5
5.2 Swap Example Using Blocking Assignment 6

6 Non-Blocking Assignment **6**
6.1 Example: Non-Blocking Assignment 6
6.2 Swap Example Using Non-Blocking Assignment 7
6.3 Blocking vs. Non-Blocking 7

7 Conditional Statements **8**
7.1 The if Statement 8
7.2 The if-else if Statement 8

8 The case Statement **9**
8.1 Example: Multiplexer Using case 9

9 Looping Statements **9**
9.1 The for Loop 9
9.2 The repeat Loop 10
9.3 The while Loop 10

10 Event Control **11**
10.1 Positive Edge 11
10.2 Negative Edge 11
10.3 Sensitivity to Any Change 11

11 Procedural Modeling of Combinational Logic **11**

11.1 Example: Half Adder 11
11.2 Half Adder Truth Table 12

12 Procedural Modeling of Sequential Logic 12
12.1 Example: D Flip-Flop 12
12.2 Example: D Flip-Flop with Asynchronous Reset 12

13 Example: Counter Using Procedural Modeling 13
13.1 Four-Bit Up Counter 13

14 Latch Inference in Procedural Blocks 13
14.1 Example: Incomplete Assignment 13
14.2 Avoiding Unintended Latches 14

15 Tasks and Functions 14
15.1 Function Example 14
15.2 Task Example 15

16 Testbench for a D Flip-Flop 15

17 Procedural Modeling Guidelines 16

18 Procedural Modeling Limitations 16

19 Procedural Modeling vs. Continuous Assignment 17

20 Summary 17

1 Introduction

Procedural modeling is a modeling technique in Verilog HDL in which circuit behavior is described using procedural statements executed in a defined simulation order.

It is commonly used to model combinational logic, sequential logic, counters, registers, finite-state machines, and testbench behavior.

Procedural statements are written inside procedural blocks such as `initial` and `always`.

The main features of procedural modeling are:

- Conditional statements using `if-else`.
- Selection statements using `case`.
- Iteration using `for`, `while`, and `repeat`.
- Blocking and non-blocking assignments.
- Event-controlled execution.
- Modeling of combinational and sequential logic.

Procedural modeling is one of the most commonly used techniques in RTL design.

2 Basic Procedural Syntax

The general syntax of a procedural block is:

```
initial begin
    // Statements execute once
end

always begin
    // Statements execute repeatedly
end
```

An `initial` block starts at simulation time zero and executes once.

An `always` block repeats its statements whenever execution reaches the end of the block.

In synthesizable RTL, an `always` block typically uses an event control or a sensitivity list to describe combinational or sequential behavior.

3 The initial Block

The `initial` block executes once during simulation. It is widely used for testbench stimulus and initialization.

3.1 Example: Initial Block

```
module initial_example;

    reg a, b;

    initial begin
        a = 0;
        b = 0;

        #10 a = 1;
        #10 b = 1;
        #10 $finish;
    end

    initial begin
        $monitor("time=%0t a=%b b=%b",
                 $time, a, b);
    end

endmodule
```

Explanation:

- Initially, both signals are assigned zero.
- After 10 time units, `a` becomes one.
- After another 10 time units, `b` becomes one.
- The simulation ends after another 10 time units.

The `$monitor` task prints the signals whenever one of its arguments changes.

An `initial` block is generally used for testbench behavior rather than ordinary synthesizable ASIC RTL.

4 The always Block

The `always` block executes repeatedly. Its behavior depends on the statements and event controls contained within it.

4.1 Example: Always Block

```
module always_example;

    reg clk;

    initial clk = 0;

    always #5 clk = ~clk;

    initial begin
        #100 $finish;
    end

endmodule
```

The clock changes state every five time units, producing a clock period of ten time units.

This is a common way to generate a clock in a simulation testbench.

For synthesizable clocked logic, an `always` block normally describes how registers respond to clock edges.

5 Blocking Assignment

Blocking assignment uses the `=` operator.

A blocking assignment evaluates its right-hand side and updates its left-hand side before the next statement in the same procedural flow executes.

5.1 Example: Blocking Assignment

```
module blocking_example;

    reg a, b, c;

    initial begin
        a = 1'b1;
        b = 1'b0;
        c = a & b;

        $display("a=%b b=%b c=%b", a, b, c);
    end
endmodule
```

The value of `c` is zero because the assignment uses the current values of `a` and `b`.

5.2 Swap Example Using Blocking Assignment

```
module blocking_swap;

    reg a, b;

    initial begin
        a = 0;
        b = 1;

        a = b;
        b = a;

        $display("a=%b b=%b", a, b);
    end
endmodule
```

Both signals become one. The first assignment updates `a`, so the second assignment reads the updated value of `a`.

Blocking assignments are commonly used for combinational logic in synthesizable RTL.

6 Non-Blocking Assignment

Non-blocking assignment uses the `<=` operator.

The right-hand side is evaluated when the statement executes, while the left-hand side update is scheduled for a later simulation event region.

This behavior is particularly useful for modeling clocked registers.

6.1 Example: Non-Blocking Assignment

```
module nonblocking_example;

    reg a, b, c;

    initial begin
        a = 1'b1;
        b = 1'b0;

        c <= a & b;

        #1;
        $display("a=%b b=%b c=%b", a, b, c);
    end

endmodule
```

The value of `c` becomes zero because the right-hand side evaluates to `1'b0`.

The delay ensures that the scheduled update has occurred before the display statement.

6.2 Swap Example Using Non-Blocking Assignment

```
module nonblocking_swap;

    reg a, b;

    initial begin
        a = 0;
        b = 1;

        a <= b;
        b <= a;

        #1;
        $display("a=%b b=%b", a, b);
    end

endmodule
```

After the scheduled updates, `a` becomes one and `b` becomes zero. Both right-hand sides are evaluated before the updates take effect.

6.3 Blocking vs. Non-Blocking

Feature	Blocking (=)	Non-Blocking (<=)
Update behavior	Updates before the next statement in the same procedural flow.	Schedules the left-hand side update.
Typical usage	Combinational logic.	Clocked sequential logic.
Execution	Statements execute in procedural order.	Right-hand sides are evaluated before scheduled updates take effect.
Common mistake	Unintended ordering dependencies.	Using it indiscriminately in combinational logic.

Design guideline: Use blocking assignments for combinational procedural logic and non-blocking assignments for clocked sequential logic. This convention helps avoid simulation race conditions.

7 Conditional Statements

Conditional statements select which statements execute based on a condition.

7.1 The if Statement

```
module if_example;

    reg a, b, y;

    always @(*) begin
        if (a)
            y = b;
        else
            y = 1'b0;
    end

endmodule
```

The output follows `b` when `a` is one. Otherwise, the output is zero.

The `@(*)` sensitivity list causes the block to execute when any signal read by the block changes.

7.2 The if-else if Statement

```
module priority_encoder_2bit (a, b, y);

    input a, b;
    output reg [1:0] y;

    always @(*) begin
        if (a)
            y = 2'b10;
        else if (b)
            y = 2'b01;
        else
            y = 2'b00;
    end

endmodule
```

This example gives priority to input `a`. If both inputs are one, the output is `2'b10`.

All branches assign `y`, so the block describes combinational logic rather than a latch.

8 The case Statement

The `case` statement selects one branch based on an expression's value.

8.1 Example: Multiplexer Using case

```
module mux4to1 (y, d, sel);

    input [3:0] d;
    input [1:0] sel;
    output reg y;

    always @(*) begin
        case (sel)
            2'b00: y = d[0];
            2'b01: y = d[1];
            2'b10: y = d[2];
            2'b11: y = d[3];
            default: y = 1'b0;
        endcase
    end

endmodule
```

The select input determines which data bit appears at the output.

The `default` branch provides an assignment for values not matched by the listed branches.

In ordinary Verilog, `case` compares four-state values exactly. `casez` treats Z and question-mark bits as wildcards, while `casex` treats X and Z as wildcards. Use `casex` cautiously because it can hide unknown-value problems.

9 Looping Statements

Verilog supports several looping constructs inside procedural blocks.

9.1 The for Loop

```
module for_loop_example;

    integer i;

    initial begin
        for (i = 0; i < 5; i = i + 1) begin
            $display("i=%0d", i);
        end
    end

endmodule
```

The loop executes five times, with `i` taking the values zero through four.

9.2 The repeat Loop

```
module repeat_example;

    reg clk;

    initial begin
        clk = 0;

        repeat (10) begin
            #5 clk = ~clk;
        end

        $finish;
    end

endmodule
```

The loop toggles the clock ten times, with a delay of five time units between toggles.

9.3 The while Loop

```
module while_example;

    integer i;

    initial begin
        i = 0;

        while (i < 5) begin
            $display("i=%0d", i);
            i = i + 1;
        end
    end

endmodule
```

The loop continues while its condition is true.

Loops are useful for testbench stimulus and for describing repetitive hardware structures where the synthesis tool supports the construct.

10 Event Control

Event control specifies when a procedural statement or block should execute.

10.1 Positive Edge

The `posedge` keyword detects a positive transition, normally from zero to one.

```
always @(posedge clk) begin
    q <= d;
end
```

This code describes a positive-edge-triggered register.

10.2 Negative Edge

The `negedge` keyword detects a negative transition, normally from one to zero.

```
always @(negedge clk) begin
    q <= d;
end
```

This code describes a register that captures its input on the falling edge of the clock.

10.3 Sensitivity to Any Change

```
always @(*) begin
    y = (a & b) | c;
end
```

The `@(*)` form is used for combinational procedural logic. It automatically includes signals read by the block in its sensitivity list.

11 Procedural Modeling of Combinational Logic

Combinational logic has outputs that depend only on the current input values.

11.1 Example: Half Adder

```
module half_adder (a, b, sum, carry);

    input a, b;
    output reg sum, carry;

    always @(*) begin
        sum = a ^ b;
        carry = a & b;
    end

endmodule
```

The sum output is the XOR of the inputs, and the carry output is their AND.

Both outputs are assigned on every execution of the block, so no storage is inferred.

11.2 Half Adder Truth Table

a	b	sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

12 Procedural Modeling of Sequential Logic

Sequential logic stores information. Its output depends on stored state and input conditions.

12.1 Example: D Flip-Flop

```
module d_flip_flop (clk, d, q);

    input clk, d;
    output reg q;

    always @(posedge clk) begin
        q <= d;
    end

endmodule
```

At every rising edge of the clock, the value of `d` is captured into `q`.

The non-blocking assignment is the conventional choice for modeling clocked registers.

12.2 Example: D Flip-Flop with Asynchronous Reset

```
module dff_async_reset (clk, rst_n, d, q);

    input clk, rst_n, d;
    output reg q;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            q <= 1'b0;
        else
            q <= d;
    end

endmodule
```

The reset is active low.

When `rst_n = 0`, the output is reset to zero without waiting for a clock edge. Otherwise, the input is captured on the rising clock edge.

13 Example: Counter Using Procedural Modeling

A counter is a common example of sequential logic described using an `always` block.

13.1 Four-Bit Up Counter

```
module up_counter (clk, rst, count);

    input clk, rst;
    output reg [3:0] count;

    always @(posedge clk or posedge rst) begin
        if (rst)
            count <= 4'b0000;
        else
            count <= count + 1'b1;
    end

endmodule
```

The counter increments on each rising clock edge and resets to zero when `rst` is one.

Since `count` is four bits wide, it wraps from 15 back to zero.

14 Latch Inference in Procedural Blocks

A latch may be inferred when a combinational procedural block does not assign an output under every possible execution path.

14.1 Example: Incomplete Assignment

```
module latch_example (en, d, q);

    input en, d;
    output reg q;

    always @(*) begin
        if (en)
            q = d;
    end

endmodule
```

When `en = 1`, the output follows `d`. When `en = 0`, no new assignment is made, so the previous value must be retained.

This describes a level-sensitive latch.

14.2 Avoiding Unintended Latches

```
module no_latch (en, d, q);

    input en, d;
    output reg q;

    always @(*) begin
        if (en)
            q = d;
        else
            q = 1'b0;
    end

endmodule
```

In this version, `q` is assigned in both branches, so no latch is required.

The intended behavior should determine whether a latch is appropriate. If storage is required, describe it explicitly and verify the synthesis results.

15 Tasks and Functions

Verilog provides tasks and functions to organize reusable procedural code.

15.1 Function Example

```
module function_example;

    function [3:0] add4;
        input [3:0] a, b;
        begin
            add4 = a + b;
        end
    endfunction

    reg [3:0] x, y, z;

    initial begin
        x = 4'd5;
        y = 4'd3;
        z = add4(x, y);

        $display("z=%0d", z);
    end

endmodule
```

The function returns the four-bit sum through its function name.

15.2 Task Example

```
module task_example;

    task show_value;
        input [3:0] value;
        begin
            $display("Value = %0d", value);
        end
    endtask

    initial begin
        show_value(4'd5);
        show_value(4'd10);
    end

endmodule
```

A task can contain multiple procedural statements and is useful for reusable testbench operations. Traditional Verilog tasks may consume simulation time through delays or event controls, while ordinary Verilog functions execute without consuming simulation time.

16 Testbench for a D Flip-Flop

A testbench applies inputs to a design and observes its response.

```
module dff_testbench;

    reg clk, d;
    wire q;

    d_flip_flop dut (
        .clk(clk),
        .d(d),
        .q(q)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        d = 0;

        #12 d = 1;
        #10 d = 0;
        #10 d = 1;

        #20 $finish;
```

```
25      end
26
27      initial begin
28          $monitor("time=%0t clk=%b d=%b q=%b",
29                   $time, clk, d, q);
30      end
31
32  endmodule
```

The testbench generates a clock with a period of ten time units and changes the input data at specified times.

The output updates on rising clock edges.

17 Procedural Modeling Guidelines

- Use `initial` primarily for testbench stimulus and simulation setup.
- Use `always @(*)` for combinational logic in traditional Verilog.
- Use `always @(posedge clk)` for positive-edge-triggered sequential logic.
- Use non-blocking assignments for clocked registers.
- Use blocking assignments for combinational procedural calculations.
- Assign every combinational output on every possible execution path.
- Use `case` statements for selection logic and finite-state machines.
- Include appropriate reset behavior when required by the design specification.
- Avoid unintended latches and simulation race conditions.
- Verify synthesizability and simulation behavior using the target toolchain.

18 Procedural Modeling Limitations

- Incorrect assignment styles can lead to simulation race conditions.
- Incomplete combinational assignments can infer unintended latches.
- Incorrect sensitivity lists in traditional Verilog can cause simulation behavior to differ from intended hardware.
- Not every procedural construct is synthesizable.
- Delays and testbench system tasks are generally not synthesizable as hardware logic.
- Complex nested conditions can make designs difficult to read and verify.

19 Procedural Modeling vs. Continuous Assignment

Feature	Procedural Modeling	Continuous Assignment
Construct	`initial`, `always`	`assign`
Execution	Statements execute within procedural blocks.	Expression continuously drives a net.
Typical usage	Combinational and sequential logic.	Simple combinational logic.
Assignment	Blocking or non-blocking.	Continuous net assignment.
Example	`always @(*) y = a & b;`	`assign y = a & b;`

Both approaches can describe combinational hardware. Procedural blocks also support clocked sequential logic and more complex control structures.

20 Summary

- Procedural modeling describes behavior using procedural statements.
- `initial` executes once, while `always` repeats execution.
- Blocking assignments use `=`.
- Non-blocking assignments use `<=`.
- Conditional logic uses `if-else` and `case`.
- Loops include `for`, `while`, and `repeat`.
- Event controls describe when procedural blocks execute.
- Combinational logic should assign outputs on every execution path.
- Sequential logic commonly uses non-blocking assignments at clock edges.
- Procedural modeling is fundamental to RTL design and testbench development.

Key takeaway: Use procedural modeling to describe combinational logic, sequential logic, and simulation behavior. Choose the appropriate assignment operator, event control, and statement structure to produce predictable, synthesizable RTL.